

第7章 下推自动机

- 7.1 基本定义
 - 下推自动机
 - PDA M的即时描述
 - PDA M用终态接受的语言、PDA M用空栈接受的语言
 - 确定的PDA
- 7.2 PDA与CFG等价
 - 7.2.1 PDA用空栈接受和用终止状态接受等价
 - 7.2.2 PDA与CFG等价

下推自动机 Pushdown Automaton

- 上下文无关语言有一种自动机来定义它们，这种自动机称为“下推自动机”，简称PDA。PDA本质上是一个 ϵ -NFA，增加了一个栈，它只能在顶部读取、压栈和弹栈，就像“栈”数据结构一样。The context-free languages have a type of automaton that defines them, this automaton is called “pushdown automaton”, referred to as PDA. PDA is essentially an ϵ -NFA with the addition of a stack which can be read, pushed, and popped only at the top, just like the "stack" data structure.
- 在本章中，我们将定义 PDA 的两种不同版本：一种通过进入终态来接受句子，就像 FA 一样，另一种通过空栈来接受句子，而不管它处于何种状态。In this chapter, we will define two different versions of the PDA: one that accepts by entering an accepting state, like FA do, and another version that accepts by emptying its stack, regardless of the state it is in.

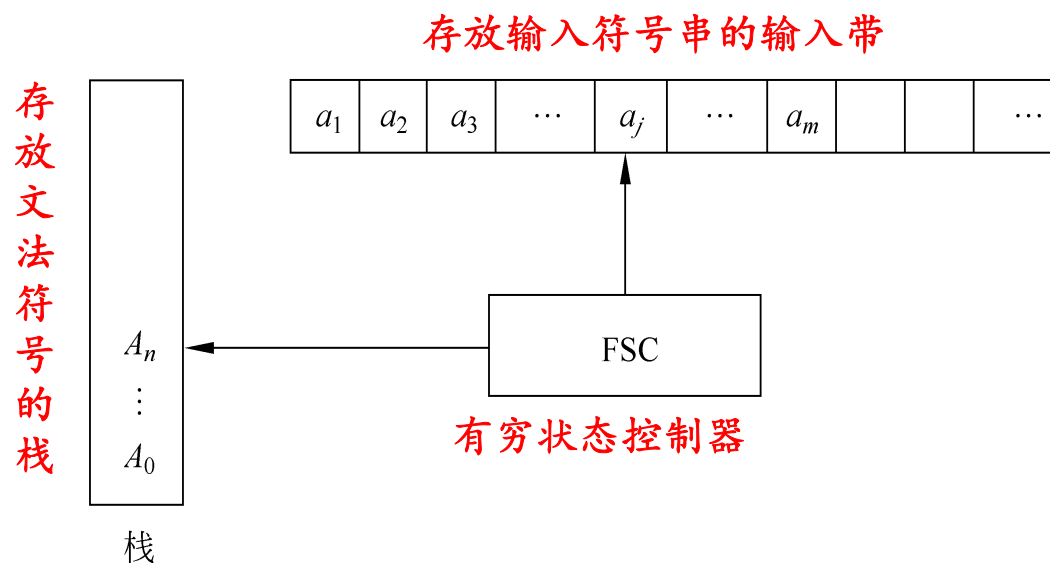
下推自动机

- 每个CFG都可以化成与其等价的GNF文法。GNF文法的产生式形如： $A \rightarrow a\alpha$ ，其中， $A \in V$ ， $a \in T$ ， $\alpha \in V^*$ 。
- 例如：有如下GNF文法
 - $S \rightarrow aAA|bBB$
 - $A \rightarrow aB|bA|a|b$
 - $B \rightarrow bB|b$
- 可以看到，在实施最左派生的过程中，句型中变量构成的子串一定会作为句型的后缀出现。
 - $S \Rightarrow a\underline{A}A \Rightarrow aa\underline{B}A \Rightarrow aab\underline{B}A \Rightarrow aabb\underline{A} \Rightarrow aabba$

- 考虑引入一个栈来存放这个变量后缀，最左边的变量最先分析，可以放在栈的最上面；最右边的变量最后分析，放在栈的最下面。
- 分析开始时，只有文法的开始符号在栈中，在分析过程中，一旦栈空，句子就被产生了。

上下文无关语言的识别模型

在有穷状态控制器的控制下，根据它的
当前状态、栈顶符号、以及输入符号作出相应的动作。
动作分为两类：



第一类动作：

根据有穷状态控制器的当前状态、栈顶符号**和输入符号**选择动作：

- 将状态改变为新状态；
- 修改栈顶——用新的语法符号串代替当前的栈顶符号；
- **向右移动读头，使读头指向下一个输入符号；**

第二类动作：

根据有穷状态控制器的当前状态、栈顶符号选择动作，**不考虑输入符号**：

- 将状态修改为新的状态；
- 修改栈顶——用新的语法符号串代替当前的栈顶符号。
- **不移动读头，是一次空移动。**

下推自动机 Pushdown Automaton

- 下推自动机(pushdown automaton, PDA)

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- Q ——状态的非空有穷集合。 $\forall q \in Q$, q 称为 M 的一个状态(state);
- Σ ——输入字母表(input alphabet)。要求 M 的输入字符串都是 Σ 上的字符串;
- Γ ——栈符号表(stack alphabet)。 $\forall A \in \Gamma$, 叫做一个栈符号;
- Z_0 —— $Z_0 \in \Gamma$, 称做开始符号(start symbol), 是 M 启动时候栈内唯一的一个符号。所以, 习惯地称其为栈底符号;
- q_0 —— $q_0 \in Q$, 是 M 的开始状态(initial state), 也可叫做初始状态或者启动状态;
- F —— $F \subseteq Q$, 是 M 的终止状态(final state)集合, 简称为终态集。 $\forall q \in F$, q 称为 M 的终止状态, 也可称为接受状态(accept state), 简称为终态。

PDA的状态转移函数

- δ —— 状态转移函数 (transition function), 又叫做状态转换函数或者移动函数。

$$\delta: \underbrace{Q}_{\text{当前状态}} \times \underbrace{(\Sigma \cup \{\varepsilon\})}_{\text{输入符号}} \times \underbrace{\Gamma}_{\text{栈顶符号}} \rightarrow 2^{Q \times \Gamma^*}$$

$$\gamma_i = A B C$$

非空移动

$$\delta(q, a, Z) = \{(p_1, \gamma_1), (p_2, \gamma_2), \dots, (p_m, \gamma_m)\}$$

表示M在状态q, 栈顶符号为Z时, 读入字符a, 对于 $i=1, 2, \dots, m$, 可以选择地将状态变成 p_i , 并将栈顶符号Z弹出, 将 γ_i 中的符号从右到左依次压入栈, 然后将读头向右移动一个带方格而指向输入字符串的下一个字符。

空移动

$$\delta(q, \varepsilon, Z) = \{(p_1, \gamma_1), (p_2, \gamma_2), \dots, (p_m, \gamma_m)\}$$

表示M进行一次 ε -移动(空移动), 即M在状态q, 栈顶符号为Z时, 无论输入符号是什么, 对于 $i=1, 2, \dots, m$, 可以选择地将状态变成 p_i , 并将栈顶符号Z弹出, 将 γ_i 中的符号从右到左依次压入栈, 读头不移动。



符号使用约定

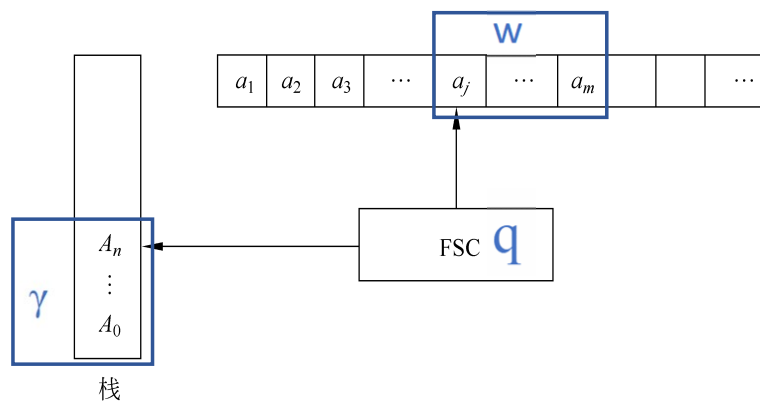
- 英文字母表较为前面的小写字母，如a, b, c, ..., 表示输入符号；
- 英文字母表较为后面的小写字母，如x, y, z, ..., 表示由输入字符串；
- 英文字母表的大写字母A, B, C, ..., 表示栈符号；
- 希腊字母 α , β , γ , ..., 表示栈符号串。

即时描述

- 即时描述(instantaneous description, ID)

$(q, w, \gamma) \in (Q, \Sigma^*, \Gamma^*)$ 称为M的一个即时描述。它表示：

- M处于状态 q
- w 是当前还未处理的输入字符串，而且M正注视着 w 的首字符
- 栈中的符号串为 γ ， γ 的最左符号为栈顶符号，最右符号为栈底的符号，较左的符号在栈的较上面，较右的符号在栈的较下面。



即时描述PDA的移动

- 如果 $(p, \gamma) \in \delta(q, a, Z)$, $a \in \Sigma$, 则

$$(q, aw, Z\beta) \vdash_M (p, w, \gamma\beta)$$

表示M做一次非空移动, 从ID(q, aw, Zβ)变成ID(p, w, γβ)。

- 如果 $(p, \gamma) \in \delta(q, \varepsilon, Z)$, 则

$$(q, w, Z\beta) \vdash_M (p, w, \gamma\beta)$$

表示M做一次空移动, 从ID(q, aw, Zβ)变成ID(p, w, γβ)。

$\vdash_M^n$: 经过n次移动

$$(q_1, w_1, \beta_1) \vdash_M^n (q_n, w_n, \beta_n)$$

$\vdash_M^*$: 经过若干次移动 (包括0次)

$$(q, w, \alpha) \vdash_M^* (p, x, \beta)$$

$\vdash_M^+$: 经过至少一次移动

$$(q, w, \alpha) \vdash_M^+ (p, x, \beta)$$

PDA接受的语言

- 下推自动机M接受的语言（有两种接受方法）

- M用终态接受的语言

$$L(M) = \{w \mid (q_0, w, Z_0) \vdash^* (p, \varepsilon, \beta) \text{ 且 } p \in F\}$$

读完输入字符串后，到达接受状态， β 可以是任意栈符号串

- M用空栈接受的语言

$$N(M) = \{w \mid (q_0, w, Z_0) \vdash^* (p, \varepsilon, \varepsilon)\}$$

读完输入字符串后，栈符号为空，此时 p 可以是任意状态

例 7-1 构造PDA 解法1

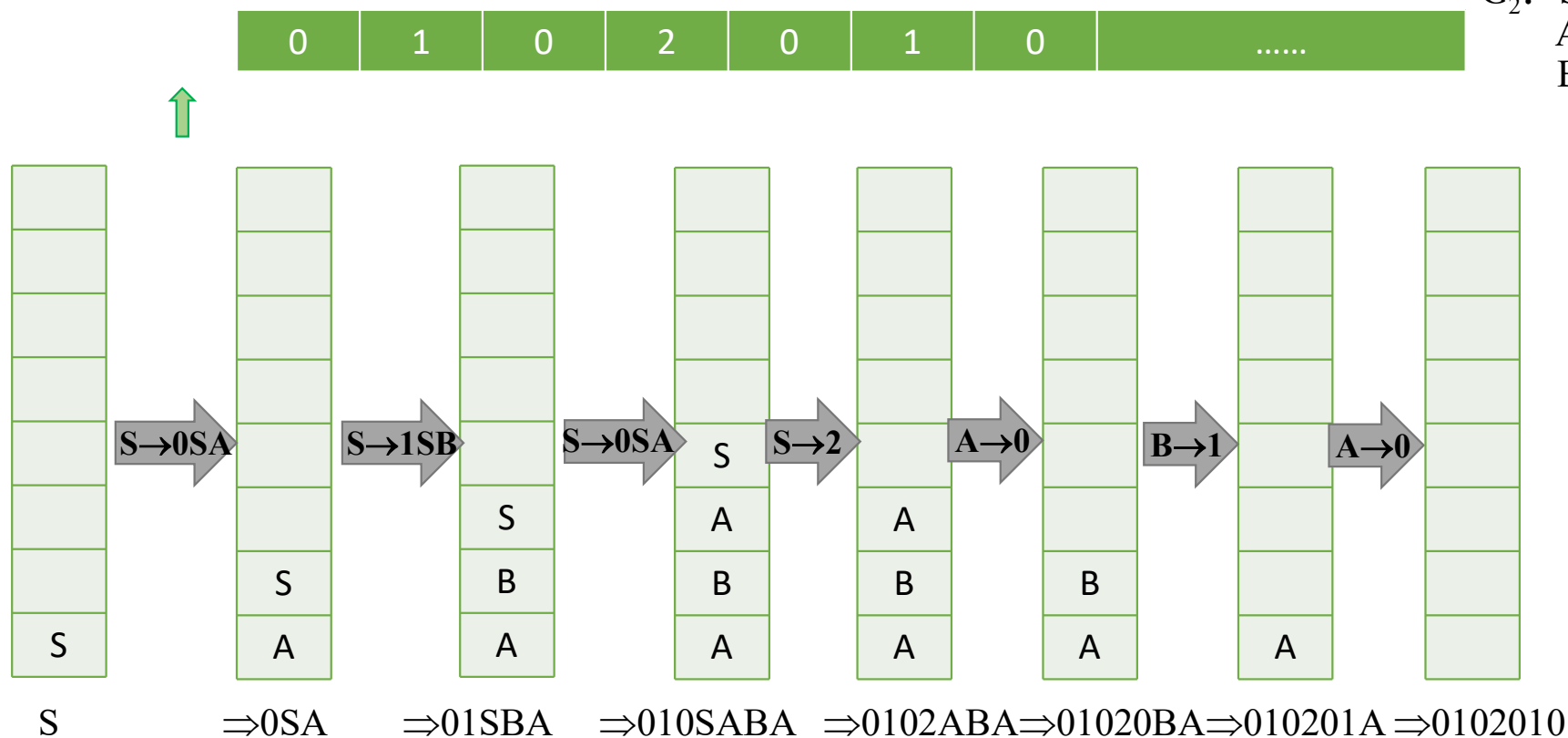
- 例 7-1 考虑接受语言 $L = \{w^2w^T \mid w \in \{0,1\}^*\}$ 的PDA的设计。
- 解法1: (先构造出相应的文法, 然后再根据文法构造出下推自动机)
- 第一步: 先设计产生L的CFG G_1 :
 $G_1: S \rightarrow 2 \mid 0S0 \mid 1S1$
- 第二步: 再将此文法转化成GNF:
 $G_2: S \rightarrow 2 \mid 0SA \mid 1SB$
 $A \rightarrow 0$
 $B \rightarrow 1$

例 7-1 构造PDA 解法1 模拟GNF最左派生

分析：句子0102010的最左派生和我们希望相应的PDA M的动作。

$$L = \{w2w^T \mid w \in \{0,1\}^*\}$$

$$G_2: \begin{array}{l} S \rightarrow 2 \mid 0SA \mid 1SB \\ A \rightarrow 0 \\ B \rightarrow 1 \end{array}$$



例 7-1 构造PDA 解法1

$G_2: S \rightarrow 2|0SA|1SB$
 $A \rightarrow 0$
 $B \rightarrow 1$

- 句子0102010的最左派生和我们希望相应的PDA M的动作。

派生	M应该完成的动作
$S \Rightarrow 0SA$	从 q_0 启动, 读入0, 将S弹出栈, 将SA压入栈, 状态不变
$\Rightarrow 01SBA$	在状态 q_0 , 读入1, 将S弹出栈, 将SB压入栈, 状态不变
$\Rightarrow 010SABA$	在状态 q_0 , 读入0, 将S弹出栈, 将SA压入栈, 状态不变
$\Rightarrow 0102ABA$	在状态 q_0 , 读入2, 将S弹出栈, 将 ϵ 压入栈, 状态不变
$\Rightarrow 01020BA$	在状态 q_0 , 读入0, 将A弹出栈, 将 ϵ 压入栈, 状态不变
$\Rightarrow 010201A$	在状态 q_0 , 读入1, 将B弹出栈, 将 ϵ 压入栈, 状态不变
$\Rightarrow 0102010$	在状态 q_0 , 读入0, 将A弹出栈, 将 ϵ 压入栈, 状态不变

例 7-1 构造PDA 解法1 空栈方式接受

- 第三步：根据GNF文法构造PDA

$G_2: S \rightarrow 2|0SA|1SB$

$A \rightarrow 0$

$B \rightarrow 1$

考虑使用空栈接受时：

$M_1 = (\{q_0\}, \{0, 1, 2\}, \{S, A, B\}, \delta_1, q_0, S, \Phi)$ 。其中：

$\delta_1(q_0, 0, S) = \{(q_0, SA)\}$ //读到0就压栈A, S保持在栈顶

$\delta_1(q_0, 1, S) = \{(q_0, SB)\}$ //读到1就压栈B, S保持在栈顶

$\delta_1(q_0, 2, S) = \{(q_0, \epsilon)\}$ //读到2就弹出栈顶符号S

$\delta_1(q_0, 0, A) = \{(q_0, \epsilon)\}$ //读到0就弹出栈顶符号A

$\delta_1(q_0, 1, B) = \{(q_0, \epsilon)\}$ //读到1就弹出栈顶符号B

此时有： $N(M_1) = L$ 。 //处理完字符串后，如果栈清空，说明可以接受该字符串。

方法

$\delta(q, a, A) = \{(q, \gamma) \mid A \rightarrow a\gamma \in P\}$,
其中, $A \in V$, $a \in T$, $\gamma \in V^*$

例7-1拓展

- 本教材未提及的是，栈符号也可以是文法中的终极符。对于文法 G_1 :

$$S \rightarrow 2|0S0|1S1$$

- 构造其PDA:

$M' = (\{q_0\}, \{0, 1, 2\}, \{S, 0, 1\}, \delta_1, q_0, S, \Phi)$ 。其中:

$\delta_1(q_0, 0, S) = \{(q_0, S0)\}$ //读到0就压栈0, S保持在栈顶

$\delta_1(q_0, 1, S) = \{(q_0, S1)\}$ //读到1就压栈1, S保持在栈顶

$\delta_1(q_0, 2, S) = \{(q_0, \epsilon)\}$ //读到2就弹出栈顶符号S

$\delta_1(q_0, 0, 0) = \{(q_0, \epsilon)\}$ //读到0就弹出栈顶符号0

$\delta_1(q_0, 1, 1) = \{(q_0, \epsilon)\}$ //读到1就弹出栈顶符号1

此时有: $N(M') = L$ 。 //处理完字符串后, 如果栈清空, 说明可以接受该字符串。

例 7-1 构造PDA 解法1 空栈接受->终态接受

考虑使用终态接受时，PDA构造如下：

$G_2: S \rightarrow 2|0SA|1SB$
 $A \rightarrow 0$
 $B \rightarrow 1$

$M_2 = (\{q_0, q_1\}, \{0, 1, 2\}, \{S, A, B, Z, Z_0\}, \delta_2, q_0, Z_0, \{q_1\})$

$\delta_2(q_0, 0, Z_0) = \{(q_0, SAZ)\}$ //压入栈底符号Z

$\delta_2(q_0, 1, Z_0) = \{(q_0, SBZ)\}$

$\delta_2(q_0, 2, Z_0) = \{(q_1, \epsilon)\}$ //仅用于接受字符串 2

$\delta_2(q_0, 0, S) = \{(q_0, SA)\}$

$\delta_2(q_0, 1, S) = \{(q_0, SB)\}$

$\delta_2(q_0, 2, S) = \{(q_0, \epsilon)\}$ //读到2，进入匹配阶段，此时栈顶为A或B

$\delta_2(q_0, 0, A) = \{(q_0, \epsilon)\}$

$\delta_2(q_0, 1, B) = \{(q_0, \epsilon)\}$

$\delta_2(q_0, \epsilon, Z) = \{(q_1, \epsilon)\}$ //见到栈底Z，则通过空移动转到终态。

此时有： $N(M_1) = L(M_2) = L$ 。

例 7-1 构造PDA 解法2

- 解法2: (根据语言的结构, 用构造FA类似的方法直接构造PDA)
- 注意到 $L=\{w2w^T|w\in\{0,1\}^*\}$, 所以 PDA M_3 的工作可以分成两大阶段。
 - 在读到字符2之前, 为“记载”阶段: 每读到一个符号就在栈中做一次相应的记载。
 - 在读到2以后, 再读到字符时, 就应该进入“匹配”阶段: 由于栈的“先进后出”特性正好与 w^T 相对应, 所以, 用栈顶符号逐一地与输入字符匹配。

例 7-1 构造PDA 解法2

基于以上思路可以构造PDA M_3

$M_3 = (\{q_0, q_1, q_2, q_f, q_t\}, \{0, 1, 2\}, \{A, B, Z_0\}, \delta_3, q_0, Z_0, \{q_f\})$

q_0 为开始状态

q_1 为记录状态

q_2 为匹配状态

q_f 为终止状态

q_t 陷阱状态

若匹配到没有定义的转移函数，即表示句子识别失败。

$\delta_3(q_0, 0, Z_0) = \{(q_1, AZ_0)\}$ //起始格局看到0

$\delta_3(q_0, 1, Z_0) = \{(q_1, BZ_0)\}$ //起始格局看到1

$\delta_3(q_0, 2, Z_0) = \{(q_f, \epsilon)\}$ //起始格局看到2

$\delta_3(q_1, 0, A) = \{(q_1, AA)\}$ //记录阶段看到0, A压栈顶

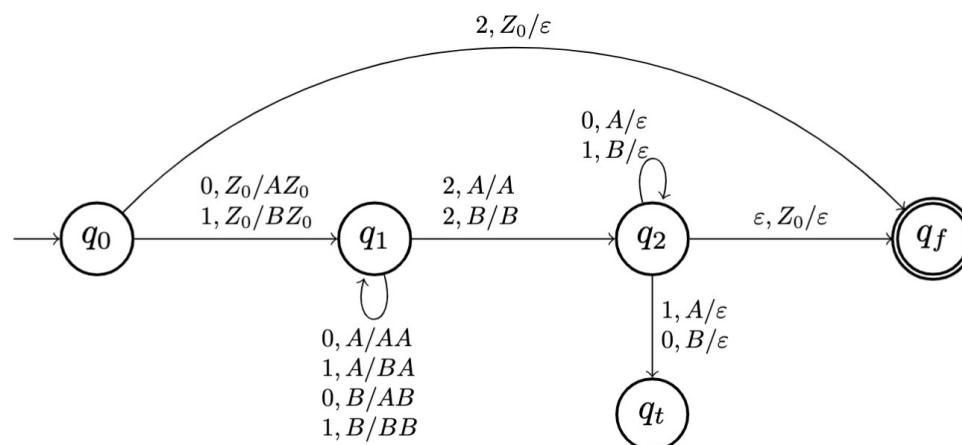
$\delta_3(q_1, 1, A) = \{(q_1, BA)\}$ //记录阶段看到1, B压栈顶

$\delta_3(q_1, 0, B) = \{(q_1, AB)\}$ //记录阶段看到0, A压栈顶

$\delta_3(q_1, 1, B) = \{(q_1, BB)\}$ //记录阶段看到1, B压栈顶

$\delta_3(q_1, 2, A) = \{(q_2, A)\}$ //栈顶为A时看到2, 进入匹配状态

$\delta_3(q_1, 2, B) = \{(q_2, B)\}$ //栈顶为B时看到2, 进入匹配状态



$\delta_3(q_2, 0, A) = \{(q_2, \epsilon)\}$ //栈顶为A时, 看到0, 匹配一个0

$\delta_3(q_2, 0, B) = \{(q_t, \epsilon)\}$ //栈顶为B时, 看到0, 进入陷阱状态

$\delta_3(q_2, 1, B) = \{(q_2, \epsilon)\}$ //栈顶为B时, 看到1, 匹配一个1

$\delta_3(q_2, 1, A) = \{(q_t, \epsilon)\}$ //栈顶为A时, 看到1, 进入陷阱状态

$\delta_3(q_2, \epsilon, Z_0) = \{(q_f, \epsilon)\}$ //看到栈底符号, 进入终止状态

此时有: $N(M_3) = L(M_3) = L$ 。

例 7-1 构造PDA 解法2

- 如果不追求让PDA同时用终止状态和空栈接受同样的语言，则可以删除状态 q_f 。这样我们可以得到PDA M_4 。
- $M_4 = (\{q_0, q_1, q_2\}, \{0, 1, 2\}, \{A, B, Z_0\}, \delta_4, q_0, Z_0, \Phi)$
- $N(M_4) = L$

$\delta_4(q_0, 0, Z_0) = \{(q_1, A)\}$	}	起始格局
$\delta_4(q_0, 1, Z_0) = \{(q_1, B)\}$		
$\delta_4(q_0, 2, Z_0) = \{(q_2, \varepsilon)\}$		
$\delta_4(q_1, 0, A) = \{(q_1, AA)\}$	}	记录阶段
$\delta_4(q_1, 1, A) = \{(q_1, BA)\}$		
$\delta_4(q_1, 0, B) = \{(q_1, AB)\}$		
$\delta_4(q_1, 1, B) = \{(q_1, BB)\}$		
$\delta_4(q_1, 2, A) = \{(q_2, A)\}$	}	阶段切换
$\delta_4(q_1, 2, B) = \{(q_2, B)\}$		
$\delta_4(q_2, 0, A) = \{(q_2, \varepsilon)\}$	}	匹配阶段
$\delta_4(q_2, 1, B) = \{(q_2, \varepsilon)\}$		

- 没有保留栈底符号。
- 扫描完字符串时，若栈底刚好弹空时，PDA接受此字符串。

确定的PDA

当我们不加任何限定词地说“PDA”时，我们指的是非确定性下推自动机。

- 定义7-4 确定的(deterministic)PDA，简称为DPDA M。

$$\forall (q, a, Z) \in Q \times \Sigma \times \Gamma,$$

$$|\delta(q, a, Z)| + |\delta(q, \varepsilon, Z)| \leq 1$$

- PDA在每一对状态 q 和栈顶符号下的动作都是惟一的。

- 理解：

- 对于 $\forall (q, Z) \in Q \times \Gamma$ ，M此时如果有非空移动，就不能有空移动。
- 每一种情况下的移动都是惟一的。

- 注意：

下推自动机与FA不同，确定的下推自动机只能接受一类特殊的CFL。也就是说，确定的下推自动机(DPDA)与一般的下推自动机是不等价的

例 7-2 构造PDA

- 例 7-2 构造接受 $L=\{ww^T|w \in \{0,1\}^*\}$ 的PDA。

$M=(\{q_0, q_1, q_f\}, \{0, 1\}, \{A, B, Z_0\}, \delta, q_0, Z_0, \{q_f\})$

q_0 : 处理句子 w 部分

q_1 : 处理句子 w^T 部分

q_f : 终止状态

例 7-2 构造PDA

$L = \{ww^T | w \in \{0,1\}^*\}$ 的PDA

$\delta(q_0, 0, Z_0) = \{(q_0, AZ_0)\}$

$\delta(q_0, 1, Z_0) = \{(q_0, BZ_0)\}$

$\delta(q_0, 0, B) = \{(q_0, AB)\}$

$\delta(q_0, 1, A) = \{(q_0, BA)\}$

$\delta(q_0, 0, A) = \{(q_0, AA), (q_1, \epsilon)\}$

$\delta(q_0, 1, B) = \{(q_0, BB), (q_1, \epsilon)\}$

$\delta(q_1, 0, A) = \{(q_1, \epsilon)\}$

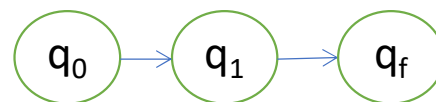
$\delta(q_1, 1, B) = \{(q_1, \epsilon)\}$

$\delta(q_1, \epsilon, Z_0) = \{(q_f, \epsilon)\}$

$\delta(q_0, \epsilon, Z_0) = \{(q_f, \epsilon)\}$ #接受 ϵ

识别句子:

1 1 0 1 0 0 1 0 1 1



A
B
A
B
B
Z ₀

回顾

- 理解PDA设计思路。
- 掌握PDA的形式化定义及工作原理。
- 理解即时描述概念。
- 理解PDA定义语言的方法。